Dennis Ritchie passes away
Dennis Ritchie, the computer scientist who pioneered the C programming language and UNIX operating system, passed away on October 13

Django: Views and Templates

Last month we learned how to get a Django web site up and running, we'll now look at the front end – the very look and feel of your new creation!

Kshitij Sobti
kshitij.sobti@thinkdigit.com

The first half of this tutorial was published in the last issue of Digit, so if you haven't read that yet, check it out at **(bit.ly/digit-django).** We've covered the installation procedure for Django and its prerequisites, and created a basic Django site too.

How Django works

It might seem a little late to cover this, but you need to understand how Django works, especially the tasks that happen between intercepting a URL, and pushing the appropriate content to the user. In Django, this happens in three steps:

1. Django matches the URL you're visiting against a number of predefined patterns. In our example, the pattern could be "book" followed by a book number as "/book/12" or "/book/9". So in our case it is "/book/<number>". Similarly, we could have "/person/number" and "/booklist" and "/personlist" all referring to different pages. Depending on which pattern is matched, the control is passed on to the view.
2. Different views are defined to deal with different types of data. A book view will take the parameter from the URL from the previous step i.e. the book number and fetch the data that needs to be displayed for this view.
3. The view will then provide this data to a template, where this data will be filled in according to the parameters specified in the template; and finally dispatched to the person viewing the page.

For example, when someone visits **http://mysite.com/book/11** Django will match it against a set of patterns, and find that it matches the `book/<number>` pattern. It will then send the book number from the URL to the view, which will pull in data corresponding to the book. This data will then be filled in as defined by a template, and the end-result will be shown to the user.

The basic Django view

STEP 1: Matching URLs

In the first part, we asked you to open the urls.py file and follow the instructions in that file to enable the admin panel. Now let us take a look at what those instructions actually did. This is what your urls.py file should be like (sans comments):

```
from django.conf.urls.defaults import patterns, include, url
from django.contrib import admin
admin.autodiscover()
urlpatterns = patterns('',
    url(r'^admin/', include(admin.site.urls)),
)
```

Pay particular attention to the urlpatterns bit of the code, since it is defining which patterns to match, and what to do with them. This bit of code tells Django that if any URL path begins with "admin/" it should immediately be passed to the admin module of Django, where it will further look at the part of the URL to the right of "admin/" and show the appropriate content.

Let us go ahead and add two more patterns here, one for the index page, and the other for an individual book page:

```
url(r'^$', 'books.views.index'),
url(r'^book/(?P<book_id>\d+)/$', 'books.views.book'),
```

The way to define patterns here is a simple semi-standard that you may have encountered before – Regular Expressions. You can refer to it at **(bit.ly/digit-regex).** You will need some knowledge of Regular Expressions for this bit.

To give a basic idea of what is happening here:

- ^ refers to the starting location, and $ refers to the ending location
- The "()" brackets around any pattern mean any condition that matches that pattern needs to be captured in a variable while the "?P<book_id>" provides the name of the variable. So a combination of the two tells the RegEx engine to capture the pattern as "book_id".
- \d+ tells the regex engine that it needs to capture one or more digits. The "\d" identifies digits, while the "+" means one or more.

The end result is that the pattern "^book/(?P<book_id>\d+)/$" will look for any URL that has a "book/" followed by a number, and if a match is found, it will capture the number part of that URL as the "book_id". This pattern will NOT match invalid URLs such as "book/12b" or "book/abc" but it will match "book/0" so you will need to check for that.

If you want to, you can have more complex patterns, which include a date or time or multiple parameters that come together to identify a resource.

The 'books.views.index' specifies the view that needs to be called in case the pattern is matched. We are yet to create this view though. This view will also be passed by any parameter captured from the URL; so in the second case, the view 'books.views.book' will be passed to the "book_id".